School of Future Tech

Case Study Report

on

Product Inventory Controller

by

Lokesh Rathod

150096725094

INDEX

1. Introduction to the Case Study
2. Problem Statement
3. Case Study Design
4. Methods, Algorithms & Technology Applied
5. Implementation Details
6. Results and Conclusion
7. References

1. Introduction to the Case Study

This project — Product Inventory Controller — is a web-based inventory management application built using HTML5, CSS3, and JavaScript.

It demonstrates Object-Oriented Programming (OOP) principles through a class based architecture that manages product stock and pricing using rule driven logic. The system is designed to simulate real world e-commerce backend and warehouse management workflows.

1.1 Objectives

- Build a fully functional inventory system using JavaScript OOP.
- Implement class based product modeling with encapsulated behavior.
- Support stock tracking, low stock detection, and bulk operations.
- Provide a clean, interactive UI with live DOM updates — no page reload.
- Design a fully responsive, dark-themed interface using HTML5 and CSS3.
- Apply input validation and error handling at every layer of the system.

2. Problem Statement

Problem Background:

Small and medium sized retail businesses frequently struggle with inventory management.

Common day to day challenges include:

- No real-time visibility into how many units of each product are in stock.
- Running out of fast selling items before the next reorder (stockouts).

- Changing prices across many products during a sale — one by one is slow and error-prone.
- Receiving bulk deliveries requires updating every product's stock manually.
- No automated warning system when a product is about to run out.

Traditional tools like spreadsheets or paper records are too slow, too manual, and offer no automation. A purpose built, rule driven inventory controller that operates in real time solves all of these problems simultaneously.

3. Case Study Design

The system is designed using a clean three layer OOP architecture. Each layer has a single, well-defined responsibility. This separation of concerns makes the code easy to understand, test, and extend.

3.1 System Architecture Overview

Layer	Component	Responsibility
Data Model	Product Class	Stores product properties; owns the rules for updating stock and price
Controller	InventoryManager Class	Manages all products in an array; handles bulk ops, search, and alerts
Bridge	Handler Functions	Read form inputs on button click, call class methods, show results
View / UI	renderTable() + DOM	Dynamically builds and injects HTML table into the page without reload

3.2 Product Class — Design Specification

i)The Product class represents a single product.

ii) It owns its own data and the rules for how that data can change.

Type	Name	Description

Property	productId	Unique identifier string — e.g. P001
Property	name	Human-readable product name — e.g. Laptop
Property	price	Current unit price in INR stored as a float
Property	stockQuantity	Number of units currently in stock — always >= 0
Method	updateStock(quantity)	Adds or subtracts qty; rejects change if result would be < 0
Method	updatePrice(newPrice)	Sets new price; validates it must be a positive number
Method	isLowStock(threshold)	Returns true if stockQuantity < threshold — used for alert detection

3.3 InventoryManager Class — Design Specification

The InventoryManager holds all product instances in a single array and provides methods that operate on the entire inventory.

Method	What It Does
addProduct(...)	Validates all inputs; creates a new Product object and pushes it to the array
findById(id)	Case-insensitive exact match search on productId using Array.find()
findByName(name)	Case-insensitive partial match search using Array.find() + includes()
findProduct(query)	Smart search: tries findById() first, falls back to findByName() using || operator
bulkPriceUpdate(percent)	Applies percentage formula to all products using Array.forEach()
bulkStockUpdate(quantity)	Safety-checks all products with Array.some() then applies update with forEach()
getLowStockProducts(threshold)	Returns filtered array of products where isLowStock() is true, via Array.filter()
getAllProducts()	Returns the full products array for table rendering
searchProducts(query)	Filters products matching partial ID or name, used by the live search bar

3.4 Data Flow Diagram

The following shows exactly what happens when any button is clicked, step by step:

Step	Layer	Action
1	HTML (View)	User fills input fields and clicks a button
2	onclick attribute	onclick='handleXxx()' triggers the corresponding JS handler function
3	Handler Function	Reads input values using document.getElementById().value
4	InventoryManager	Handler calls the relevant InventoryManager class method
5	Product Class	InventoryManager calls Product methods (updateStock, isLowStock, etc.)
6	Handler (response)	result.success checked; showMessage() displays green/red feedback
7	renderTable()	Table rebuilt in-place — DOM updated, no page reload

4. Methods, Algorithms & Technology Applied

4.1 Technology Stack

Technology	Version / Type	Role in the Project
HTML5	HTML5	Semantic page structure — sections, forms, inputs, buttons, tables
CSS3	CSS3	Dark theme, animations (fadeUp, rowIn, pulse), responsive grid layout
JavaScript	ES6+ (Vanilla)	OOP classes, DOM manipulation, event handling, state management
Google Fonts	External CDN	Syne (display headings/buttons) + Space Mono (mono inputs, tables)
Browser	Chrome/Firefox/Edge	Sole runtime environment — zero server, zero database required

4.2 OOP Concepts Applied

OOP Concept	Where in the Code	What It Achieves
Class	class Product { }	Blueprint for creating product objects
Class	class InventoryManager { }	Blueprint for the inventory system controller
Constructor	constructor(id, name, price…)	Auto-runs on new Product() to set properties
Encapsulation	updateStock(), updatePrice()	Logic and rules hidden inside the class
this keyword	this.price, this.stockQuantity	Refers to the current specific product instance
Array.forEach()	bulkPriceUpdate, bulkStock	Apply an operation to every product in the array
Array.filter()	getLowStockProducts()	Returns only products matching a condition
Array.find()	findById(), findByName()	Returns first product matching a search query
Array.some()	bulkStockUpdate() safety check	Checks if ANY product would go below 0
Template Literals	renderTable() HTML builder	Embeds JS values inside HTML strings with ${}
DOM Manipulation	container.innerHTML = html	Updates the page live without reloading
Event Handling	onclick='handleXxx()'	Runs a function when the user clicks a button

4.3 Key Algorithms

1. Low Stock Detection: Each product has an isLowStock(threshold) method that returns a simple boolean comparison. The inventory system uses Array.filter() to collect all flagged products at once.

2. Bulk Price Update Formula: A standard percentage-change formula is applied to every product in a single pass using forEach():

3.Bulk Stock Update: Before any change is applied, Array.some() is used to verify that no product would go below 0. Only if ALL products pass the check does forEach() apply the update:

4 Dynamic Table Rendering (DOM Manipulation): renderTable() builds a complete HTML table as a string using a forEach loop and template literals, then injects it directly into the page with innerHTML — replacing the old table instantly:

5. Implementation Details

5.1 File Structure and Line Count

File	Contents
index.html	5 section cards: Add Product, Update Stock, Bulk Price, Bulk Stock, Low Stock + Inventory Table + Search
style.css	Dark theme, CSS variables, fadeUp / rowIn / pulse animations, badge styles, responsive media query
script.js	Product class + InventoryManager class + 6 handler functions + renderTable() + IIFE sample data

5.2 UI Sections — Detailed Implementation

UI Section	User Inputs Required	What Happens Internally
➕ Add Product	ID, Name, Price, Stock	Validates all 4 fields, creates new Product object, pushes to array, refreshes table
🔄 Update Stock	Product ID/Name, Qty±	Finds product by ID or name, calls updateStock(), shows result, refreshes table
💰 Bulk Price	Percentage (e.g. 10)	Validates %, calls bulkPriceUpdate() on all products, refreshes table
📦 Bulk Stock	Quantity (e.g. +20/-5)	Pre-flight check with some(), then updateStock() on all, refreshes table
⚠️ Low Stock Alert	Threshold (e.g. 10)	Filters via getLowStockProducts(), renders pulsing alert cards below button
📋 Inventory Table	Search input (live)	Calls searchProducts() on every keystroke, re-renders filtered table instantly

5.3 Validation Rules Implemented

- Product ID must be non empty and unique — duplicate IDs are rejected with an error message.
- Product name is mandatory — blank name field triggers an error.
- Price must be a positive number greater than 0 — text values and 0 are rejected.
- Stock quantity must be >= 0 at creation and can never go below 0 after any update.
- Bulk stock update pre checks all products before applying any change.
- Bulk price percentage must be a valid number.
- Low stock threshold must be a non-negative number.
- All inputs use .trim() to strip accidental whitespace before validation.

6. Results and Conclusion

The Product Inventory Controller (OOP-Based) was successfully designed, developed, and tested as a fully functional browser based application. The project meets all the academic requirements of case study and demonstrates practical, applied knowledge of HTML5, CSS3, and JavaScript.

The two class OOP architecture with Product as the data model and InventoryManager as the system controller proved to be an effective, clean design. It separates responsibility clearly, makes the code readable and maintainable, and accurately models how a real world inventory system would be structured in a professional codebase.

Key Learnings:

- OOP in JavaScript allows real world entities to be cleanly modelled as classes with properties and encapsulated methods.
- Separating data (Product), logic (InventoryManager), and view (renderTable) makes code easier to debug, test, and extend.
- DOM manipulation with innerHTML and getElementById enables dynamic, no reload web applications.
- CSS animations improve user experience significantly with very little code.
- Input validation at every layer prevents invalid application states and improves system reliability.
- Pre flight checks (Array.some() before forEach) are an important pattern when performing bulk operations.

7. References

1. MDN Web Docs — JavaScript Classes (ES6). Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes
2. MDN Web Docs — CSS Animations and @keyframes. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animations
3. MDN Web Docs — Template Literals (Template Strings). Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals
4. W3Schools — JavaScript OOP and Classes Tutorial. Available at: https://www.w3schools.com/js/js_object_oriented.asp
5. W3Schools — HTML5 Forms and Input Elements. Available at: https://www.w3schools.com/html/html_forms.asp
6. W3Schools — CSS Responsive Web Design. Available at: https://www.w3schools.com/css/css_rwd_intro.asp
7. JavaScript.info — Classes (comprehensive OOP guide). Available at: https://javascript.info/classes
8. CSS-Tricks — A Complete Guide to CSS Grid. Available at: https://css-tricks.com/snippets/css/complete-guide-grid/
9. Course Material: HTML-5, CSS, JavaScript — Semester II. ITM Skills University, B.Tech CSE 2025-29.